

Niveles de paralelismo en procesadores

Agenda

- 1. Nivel de paralelismo de bits**
- 2. Nivel de paralelismo de instrucciones**
 - a. Pipelining.
 - b. Superscalar y Very Long Instruction Word (VLIW).
- 3. Nivel de paralelismo de threads**
 - a. Multi-Threading.
 - b. Simultaneous Multi-Threading.
 - c. Chip-level Multiprocessing.
- 4. Nivel de paralelismo de datos**
 - a. Array processor
 - b. Vector processor

Paralelismo de nivel de bits

Paralelismo de nivel de bits

- Cuando se dice “un procesador es de 32 bits” significa que el tamaño de la palabra es de 32 bits.
- El paralelismo de nivel de bits se basa en **incrementar el tamaño de la palabra** del procesador para reducir el número de instrucciones requeridas para operaciones sobre datos mayores a una palabra.

Paralelismo de nivel de instrucciones

Paralelismo de nivel de instrucciones (ILP)

El paralelismo de nivel de instrucciones (ILP, del inglés *Instruction Level Parallelism*) se refiere a la **ejecución en paralelo de múltiples instrucciones de un mismo flujo de instrucciones**, las cuales son independientes unas de otras.

El resultado de la ejecución del flujo de instrucciones siempre debe ser igual a la ejecución en el orden dado por el programa.

Técnicas para explotar el ILP

- Segmentado de Instrucciones.
- Pipelines múltiples:
 - Superescalar.
 - Very Long Instruction Word.
- Renombrado de registros.
- Predicción de saltos y ejecución especulativa.
- Reenvío de operandos.
- Ejecución fuera de orden.

Segmentado de instrucciones

- Se solapa parcialmente la ejecución de múltiples instrucciones.
- El ciclo de reloj se ajusta a la etapa más lenta.



Pipelines múltiples

(o procesadores de emisión múltiple)

- Estos procesadores tienen múltiples unidades funcionales independientes (ALUs, FPUs, unidades de carga y almacenamiento, unidades de salto, etc.).
- Para utilizar las múltiples unidades funcionales del procesador, se emiten varias instrucciones por ciclo (*multiple issue*).
- Se buscan instrucciones que:
 - Sean independientes
 - Usen diferentes unidades funcionales

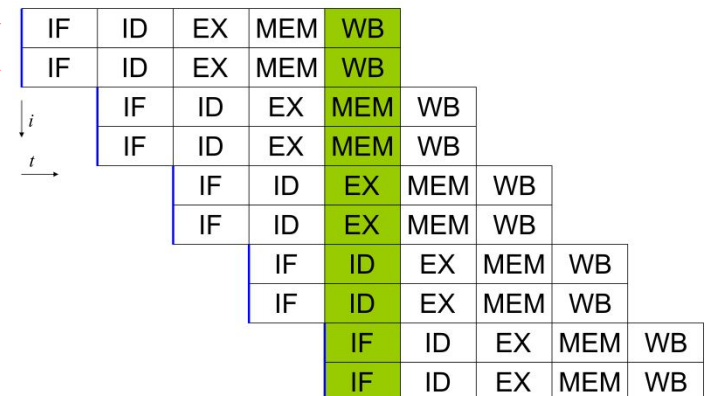
```
float a, b, c  
integer i, j, k
```

```
a = b * c  
i = j + k
```

usa la FPU

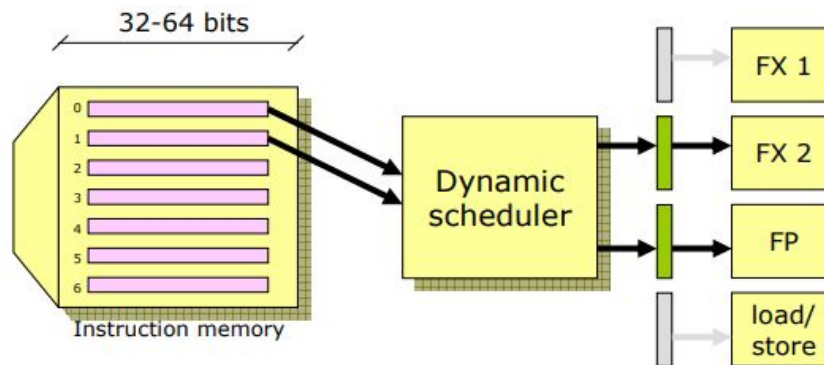
usa la ALU

Emisión múltiple

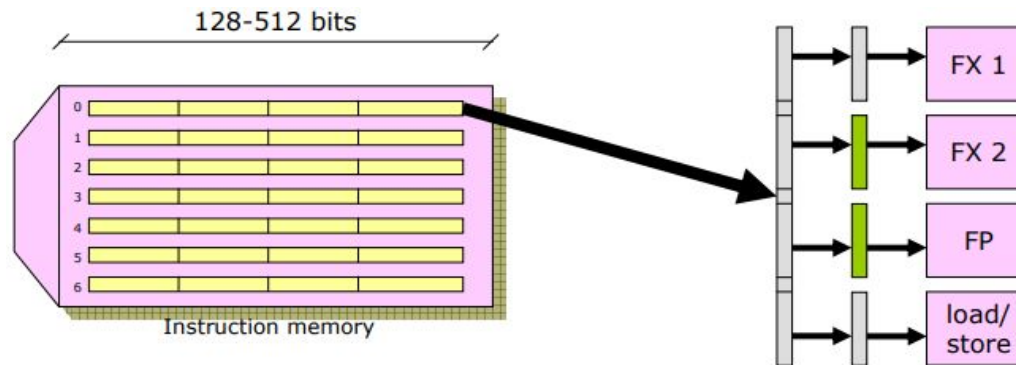


Pipelines múltiples: clasificación

- **Superescalar:** la emisión de las instrucciones las planifica el hardware de manera dinámica (*dynamic multiple issue*), en tiempo de ejecución.



- **Very Long Instruction Word (VLIW):** las instrucciones son planificadas estáticamente por el compilador (*static multiple issue*).



Problemas de los pipelines

Factores que pueden afectar el rendimiento de los pipelines:

- Conflictos de recursos
- Dependencias de datos
- Dependencias de control

Existe **riesgo** (***hazard***) cuando dos instrucciones (afectadas por alguno de los tres factores) están lo suficientemente cerca como para que la superposición de la ejecución de las instrucciones produzca un resultado inesperado. Los riesgos se clasifican en:

- Estructural (Structural Hazard)
- De datos (Data Hazard)
- De control (Control Hazard)

Problemas de los pipelines: conflicto de recursos

Dos instrucciones están en conflicto de recursos cuando requieren al mismo tiempo la misma parte del hardware del procesador.

Problemas de los pipelines: dependencias de datos

Dependencia de datos verdadera o read-after-write (RAW)

La salida (en un registro o ubicación de memoria) de una instrucción es entrada de una instrucción posterior.

```
i1      MUL R4, R3, R1    //R4 ← R3 * R1
...
i2      ADD R2, R4, R5    //R2 ← R4 + R5
```

i2 depende de i1

Riesgo: que i2 intente leer R4 antes de que i1 escriba en R4

Problemas de los pipelines: dependencias de datos

Dependencia de salida o write-after-write (WAW)

Ambas instrucciones escriben en la misma ubicación (registro o memoria). Es una dependencia de nombre.

```
i1      MUL R4, R3, R1    //R4 ← R3 * R1
...
i2      ADD R4, R2, R5    //R4 ← R2 + R5
```

i2 tiene una dependencia de salida con i1

Riesgo: que i2 intente escribir en R4 antes de que lo haga i1.

Problemas de los pipelines: dependencias de datos

Anti-dependencia o write-after-read (WAR)

Una instrucción requiere un valor (de registro o memoria) que es actualizado con posterioridad. Es una dependencia de nombre.

```
i1      MUL R4, R3, R1    //R4 ← R3 * R1
...
i2      ADD R3, R2, R5    //R3 ← R2 + R5
```

i2 anti-depende de i1

Riesgo: que i2 intente escribir en R3 antes de que i1 lea R3.

Problemas de los pipelines: dependencias de datos

Las dependencias entre registros son simples de detectar,
pero no así las dependencias entre ubicaciones de memoria.

Ejemplo:

100(R4) y 20(R6) podrían ser direcciones idénticas

Problemas de los pipelines: dependencias de control

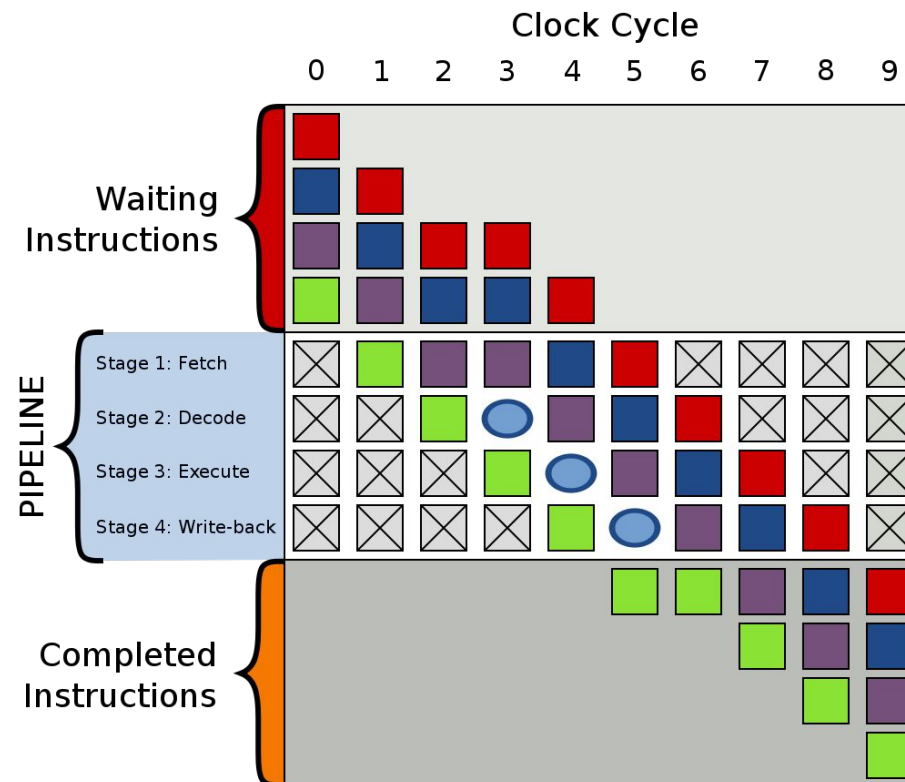
```
S1  if (a == b)
S2    a = a + b;
S3  b = a + b;
```

S2 tiene una dependencia de control con S1.

S1 y S3 no tienen dependencias de control con otras sentencias.

Pipeline bubbling / stall / break

- Se insertan burbujas en el pipeline para retrasar la ejecución de instrucciones y así resolver cualquier tipo de riesgo (estructural, de datos o control).
- Produce pérdida de rendimiento.



Renombrado de registros

Técnica utilizada para intentar solucionar las dependencias de nombres en registros.

```
i1  MUL R4,R3,R1    //R4 ← R3 * R1
...
i2  ADD R3,R2,R5     //R3 ← R2 + R5
```



```
i1  MUL R4,R3,R1    //R4 ← R3 * R1
...
i2  ADD R7,R2,R5     //R7 ← R2 + R5
```



```
i1  MUL R4,R3,R1    //R4 ← R3 * R1
...
i2  ADD R4,R2,R5     //R4 ← R2 + R5
```



```
i1  MUL R4,R3,R1    //R4 ← R3 * R1
...
i2  ADD R6,R2,R5     //R6 ← R2 + R5
```

Predicción de saltos y ejecución especulativa

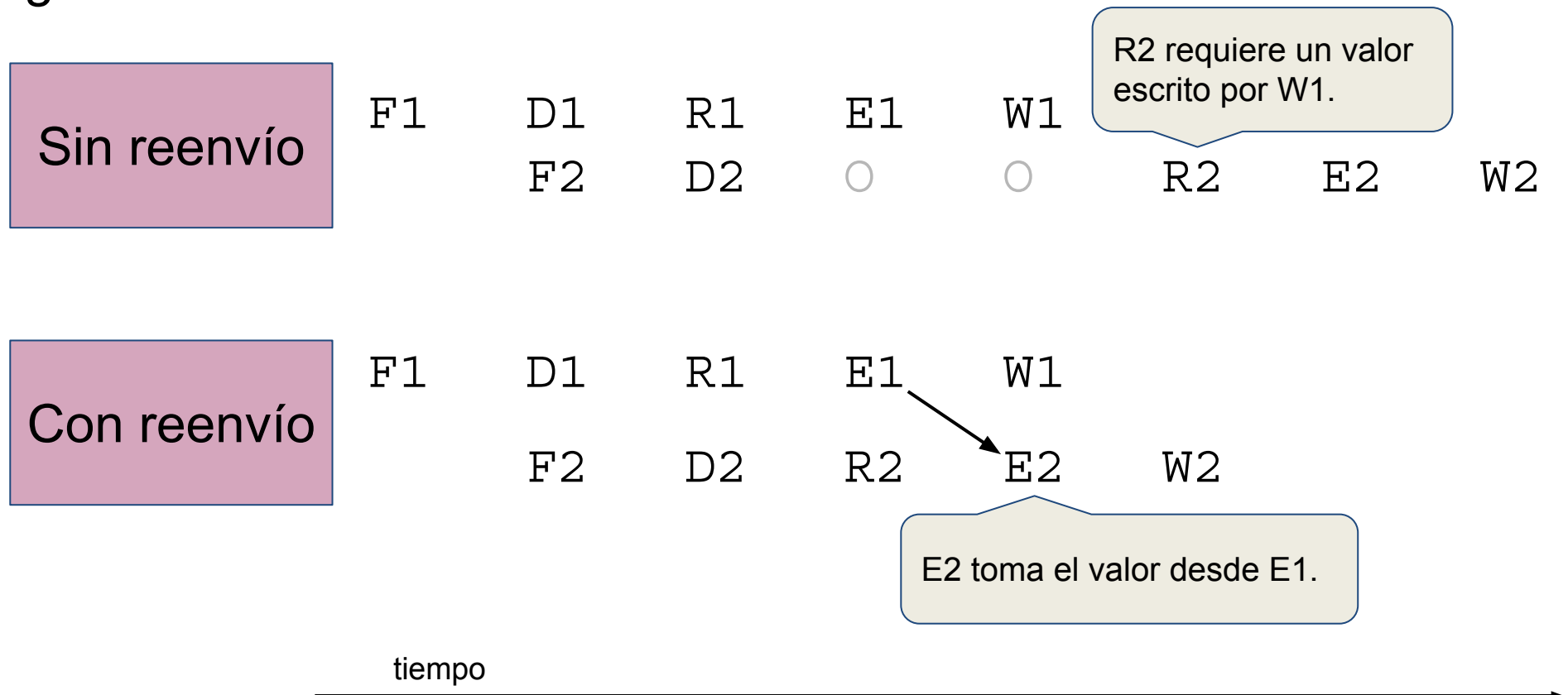
Técnicas que evitan esperas ociosas del procesador ante una dependencia de control.

Predicción de saltos: Antes de ejecutar los saltos condicionales, se intenta adivinar qué camino seguirá la ejecución, basándose en datos históricos.

Ejecución especulativa: Se ejecutan instrucciones de uno de los caminos, especulando en que luego sean útiles. ¿Qué sucede si finalmente era el camino equivocado? se vacía el pipeline.

Reenvío de operandos

Para no esperar a que un dato, que procesó una unidad funcional, sea escrito en un registro, el dato puede ser reenviado directamente desde la unidad funcional hacia la etapa de ejecución de la siguiente instrucción.



Ejecución fuera de orden

La ejecución fuera de orden ejecuta instrucciones en un orden **gobernado por la disponibilidad de los datos de entrada y unidades funcionales** en vez del orden dado por el programa.

Permite ejecutar más instrucciones en paralelo.

Ejecución fuera de orden

Ciclo de instrucción con emisión desordenada y finalización ordenada:

- 1) Búsqueda y decodificación de la instrucción.
- 2) Despacho (*dispatch*): la instrucción va a una cola de instrucciones.
- 3) Espera por operandos: la instrucción espera en la cola hasta que sus operandos estén disponibles.
- 4) Emisión: la instrucción es emitida (*issue*) a la unidad funcional apropiada para ser ejecutada.
- 5) Encolado de resultados.
- 6) Retiro: después de que instrucciones anteriores han escrito los resultados, se escriben los resultados de ella.

Ejecución fuera de orden

Reducción de detenciones por acceso al subsistema de memoria

Programa

$x = 3 * a$ ← está en caché
 $y = 2 * b$ ← está en registro

Ejecución ordenada

Pipeline 1



$x = 3 * a$
 $y = 2 * b$

Ejecución fuera de orden

Pipeline 1

$y = 2 * b$
 $x = 3 * a$

Mientras se busca a , se puede ejecutar $2 * b$.

Ejecución fuera de orden

Reducción de detenciones por dependencias de datos

Programa

```
a = b * c  
d = a * 2  
i = j + k  
m = i + 3
```

Ejecución ordenada

Pipeline 1

Pipeline 2

```
a = b * c  
d = a * 2  
i = j + k  
m = i + 3
```

Ejecución fuera de orden

Pipeline 1

Pipeline 2

```
a = b * c  
d = a * 2  
i = j + k  
m = i + 3
```

Ejecución fuera de orden

Reducción de detenciones por conflictos de recursos

El procesador tiene una única **ALU** y una única **FPU**.

Reales: b, c, e, f .

Enteros: j, k, m, n.

Programa

a = b * c

d = e * f

i = j + k

l = m + n

Ejecución ordenada

Pipeline 1

Pipeline 2

a = b * c

d = e * f

l = m + n

i = j + k

Ejecución fuera de orden

Pipeline 1

Pipeline 2

a = b * c

d = e * f

i = j + k

l = m + n

Paralelismo de nivel de threads

Paralelismo de nivel de threads

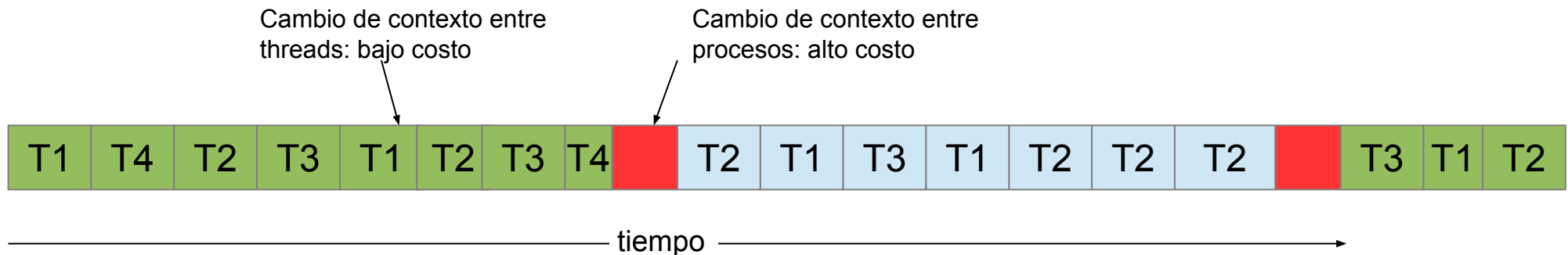
El paralelismo a nivel de threads se refiere a la **ejecución en paralelo** de instrucciones **de múltiples threads**.

Técnicas para explotar paralelismo a nivel de threads:

- Multithreading
- Multithreading simultáneo
- Multiprocesamiento de nivel de chip

Multithreading

- Las aplicaciones ejecutan múltiples threads, y los cambios de contexto entre threads son de bajo costo.



- Si un thread se bloquea, la CPU puede ejecutar instrucciones de otros threads, aumentando el uso de las unidades funcionales.
- Restricción en procesadores de múltiples pipelines: en cada nuevo ciclo de reloj se despachan instrucciones de un único thread.

Multithreading simultáneo (SMT, Simultaneous Multithreading)

Permite emitir instrucciones de varios threads (posiblemente de diferentes aplicaciones) en el mismo ciclo de reloj para aumentar el uso de las unidades funcionales.

Multithreading

IF	ID	EX	MEM	WB		
IF	ID	EX	MEM	WB		
	IF	ID	EX	MEM	WB	
	IF	ID	EX	MEM	WB	
		IF	ID	EX	MEM	WB
		IF	ID	EX	MEM	WB

Multithreading simultáneo

IF	ID	EX	MEM	WB		
IF	ID	EX	MEM	WB		
	IF	ID	EX	MEM	WB	
	IF	ID	EX	MEM	WB	
		IF	ID	EX	MEM	WB
		IF	ID	EX	MEM	WB

Multiprocesamiento de nivel de chip (Chip-level Multiprocessing)

Estos procesadores son los denominados **multicore**: se replica la CPU para soportar la ejecución simultánea de múltiples threads dentro de un único chip procesador.

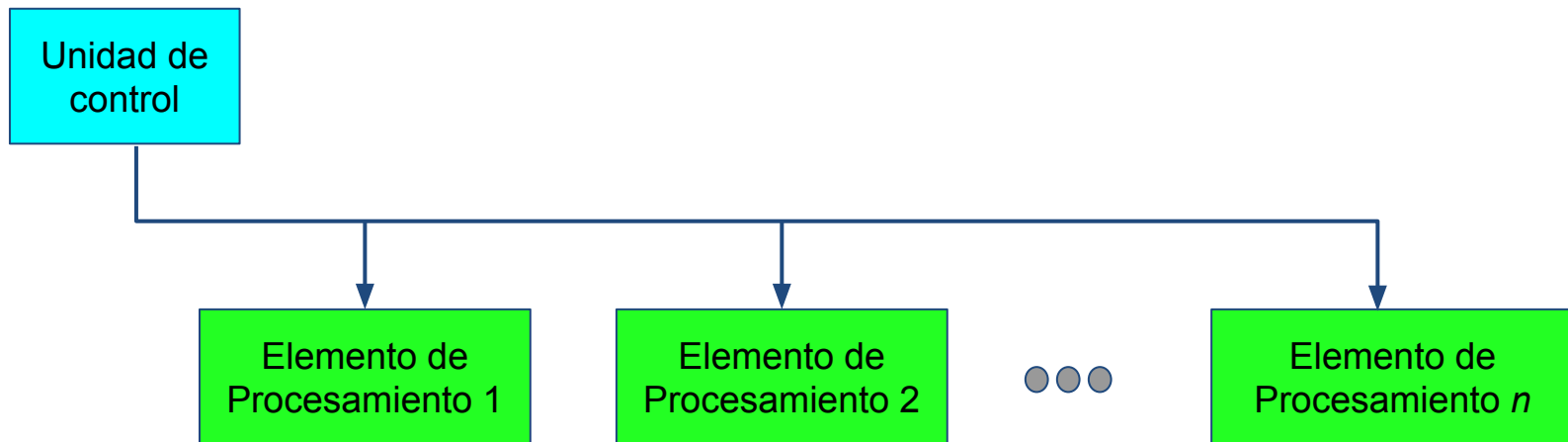
Paralelismo de nivel de datos

Paralelismo de nivel de datos (DLP)

DLP se refiere al procesamiento SIMD en dos tipos de procesadores:

- **Procesadores matriciales (array processor):** tiene múltiples elementos de procesamiento operando en paralelo sobre múltiples elementos de datos
- **Procesadores vectoriales (vector processor):** tiene un solo elemento de procesamiento operando en secuencia sobre múltiples elementos de datos

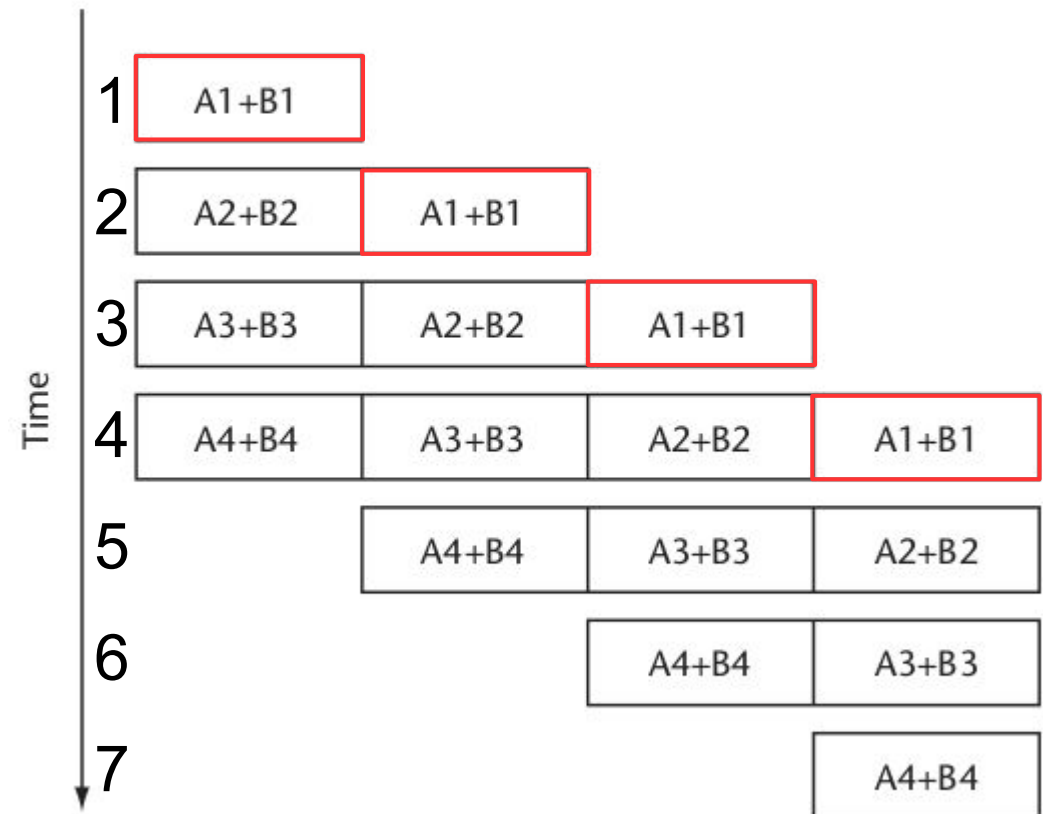
Procesador matricial



Procesador vectorial

Suma vectorial de números de coma flotante

Etapas: Align → Add → Normalize → Round



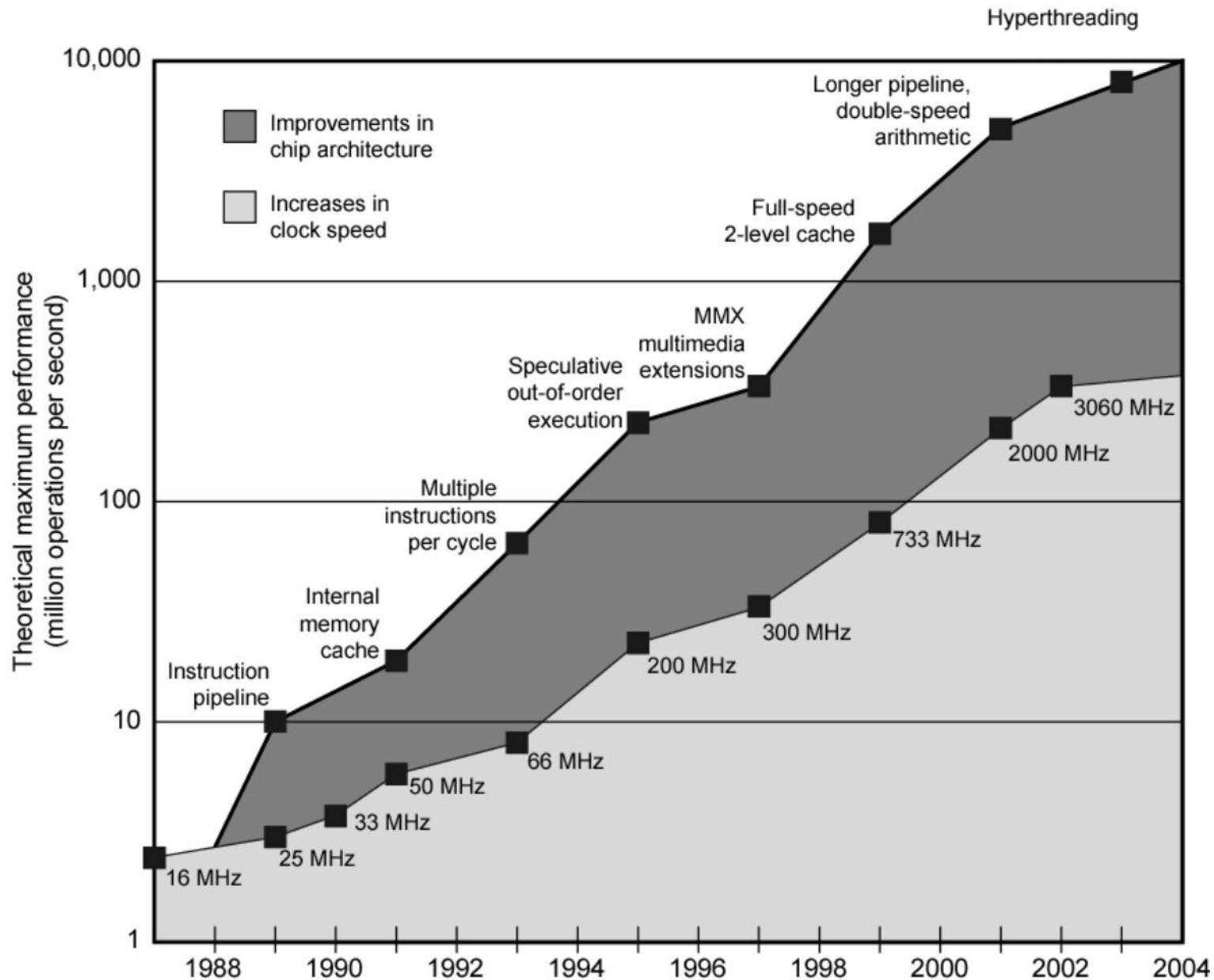
Unidad de control

Elemento de Procesamiento

Se implementa un pipeline aritmético en unidades funcionales

Origen de las mejoras del rendimiento

Origen de las mejoras del rendimiento



Bibliografía

- Del libro de Rauber:
 - 2.1 Processor Architecture and Technology Trends
- Del libro de Grama:
 - 2.1 Implicit Parallelism: Trends in Microprocessor Architectures
 - 2.1.1 Pipelining and Superscalar Execution
 - 2.1.2 Very Long Instruction Word Processors
- Del libro de Dongarra
 - 2.2.3 CPU Parallelism
- Del libro de Patterson
 - 4.10 Parallelism via Instructions
 - 6.4 Hardware Multithreading
- Del libro de Hennessy (solo si se desea una lectura en profundidad)
 - 3 Instruction-Level Parallelism and Its Exploitation
 - Appendix C. Pipelining: Basic and Intermediate Concepts.